

## Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>SYSTEM DESIGN CASE STUDY .....</b>                               | <b>2</b>  |
| <b>Introduction &amp; Executive Context .....</b>                   | <b>3</b>  |
| <b>Executive Summary .....</b>                                      | <b>3</b>  |
| <b>The Problem Statement .....</b>                                  | <b>4</b>  |
| <b>Target State Architecture &amp; Structural Design .....</b>      | <b>5</b>  |
| <b>Comprehensive Solution Summary .....</b>                         | <b>7</b>  |
| <b>End-End Customer Journey .....</b>                               | <b>9</b>  |
| <b>Transaction Data-Flows .....</b>                                 | <b>12</b> |
| <b>Security Architecture .....</b>                                  | <b>14</b> |
| <b>Disaster Recovery (DR) Plan .....</b>                            | <b>16</b> |
| <b>Architecture Tradeoffs .....</b>                                 | <b>19</b> |
| <b>Architecture Decision Records .....</b>                          | <b>21</b> |
| <b>AWS Well-Architected Framework Review .....</b>                  | <b>23</b> |
| <b>Architecture Risks .....</b>                                     | <b>26</b> |
| <b>Architectural Recommendations .....</b>                          | <b>29</b> |
| <b>Boundary Conditions for Reference Architecture .....</b>         | <b>32</b> |
| <b>Appendix A: Non-Functional Requirements (NFR) Matrix .....</b>   | <b>34</b> |
| <b>Appendix B: Cost Optimization &amp; Attribution Matrix .....</b> | <b>37</b> |
| <b>Appendix C: Enterprise Security Risk Register .....</b>          | <b>40</b> |
| <b>Appendix D: Enterprise RACI Matrix .....</b>                     | <b>43</b> |
| <b>About the Author .....</b>                                       | <b>45</b> |

## SYSTEM DESIGN CASE STUDY

### **Legacy Salesforce Hybrid Architecture on AWS**

**Prepared by:**

**Amit Kulkarni**

## Introduction & Executive Context

This architecture diagram illustrates a robust, event-driven hybrid integration model designed to seamlessly connect *Salesforce* with *Amazon Web Services* (AWS) serverless infrastructure. The system is engineered to handle bidirectional communication, processing real-time event payloads from *Salesforce* and returning execution status updates via scheduled workflows. By leveraging key AWS services such as *Amazon AppFlow*, *AWS EventBridge*, *AWS Lambda*, and *AWS DynamoDB*, the solution ensures high availability, scalability, and decoupled processing for enterprise workflows. From an executive perspective, this architecture enhances operational efficiency by automating cross-platform data synchronization, providing real-time data streaming for contact center solutions like *Amazon Connect*, and ensuring business continuity through automated error detection and proactive support notifications via *Amazon SNS*.

## Executive Summary

This comprehensive architecture establishes a highly scalable, event-driven integration framework between *Salesforce* and AWS Cloud, designed to automate bidirectional data synchronization and eliminate operational silos. At its core, the solution ingests real-time event payloads from Salesforce using an event-based *Amazon AppFlow* connector, routing them through *Amazon EventBridge* and *AWS Lambda* to write directly into *Amazon DynamoDB*. To close the feedback loop, changes in *DynamoDB* trigger *DynamoDB Streams*, which orchestrate a return path via *AWS Lambda* and *AWS S3* back to a schedule-based *Amazon AppFlow* configuration, updating Salesforce with the final execution status. This serverless ecosystem minimizes infrastructure overhead while maximizing reliability through specialized parallel workflows: one path streams data directly from *DynamoDB* to *Amazon Connect* via *Lambda* to empower customer-facing teams with real-time context, while another utilizes *DynamoDB Streams* and *Lambda* to push status alerts through *Amazon SNS* to business users. Crucially, the design ensures enterprise-grade business continuity by embedding an automated error-handling mechanism; if the primary *AppFlow* link breaks, *EventBridge* immediately catches the failure, routing it through *Lambda* and *Amazon SNS* to send instant email alerts to the AWS Solution Support Team for proactive remediation.

# The Problem Statement

## 1. Inefficient Real-Time Event Ingestion from Salesforce

- The enterprise system must capture high-volume payload data instantly from Salesforce CRM events without dropping messages.
- Legacy point-to-point connections suffer from tight coupling, leading to system timeouts and data degradation during peak business hours.
- Inbound event payloads must be validated, parsed, and decoupled using scalable middleware before reaching core database layers.
- The infrastructure requires an event-driven framework that routes inbound messages smoothly to dedicated serverless execution environments.

## 2. Delayed Bidirectional Status Reconciliation and Feedback Loops

- Business users lack immediate visibility into processing statuses, causing data discrepancies between AWS and Salesforce fields.
- Traditional batch processing for updates introduces data latency, which disrupts critical customer operations.
- System state mutations within Amazon DynamoDB must be captured instantly to trigger downstream synchronization routines.
- The architecture must safely write operational outputs and execution statuses back to Salesforce using managed schedules.

## 3. Inadequate Failure Detection and Operational Alerting Mechanisms

- Unmonitored integration breaks between Salesforce and the AWS cloud boundary result in untracked data loss.
- The cloud support team lacks automated, centralized visibility when critical serverless execution flows break down mid-transit.
- Dead-letter events must be automatically captured, isolated, and sent through messaging services to prevent pipeline blockages.
- The system requires differentiated alerting pathways to separate urgent infrastructure failures from standard business notifications.

## 4. Disconnected Downstream Omni-Channel Orchestration and Notifications

- Downstream contact center applications do not receive real-time updates from the primary transactional database layer.
- Customer service representatives operate without immediate contextual updates during active user interactions.

- Automated systems need to dynamically invoke contact center workflows based on specific state changes in the cloud ecosystem.
- Separate notification workflows must be established to simultaneously update business stakeholders and IT maintenance groups.

## Target State Architecture & Structural Design

### System Architecture Overview

The system uses an *Event-Driven Architecture* (EDA) combined with a Serverless Data Processing Pipeline. It facilitates asynchronous, bidirectional communication between a *SaaS* platform (*Salesforce*) and custom cloud-native infrastructure (*AWS Cloud*). The design focuses on decoupling system components to guarantee high availability, fault tolerance, and eventual consistency between the *CRM* and *AWS* data planes.

### Core Functional Processing Tiers

#### 1 Inbound Event Ingestion & Processing Path (Steps 1–5)

This pathway ingests real-time business data from *Salesforce* into the *AWS* core database layer.

- **Ingestion Layer (Amazon AppFlow-1):** A fully managed integration service that securely pulls inbound event payloads from *Salesforce* via automated, event-driven triggers.
- **Routing Bus (Amazon EventBridge):** Receives the payload and acts as a central message broker. It filters, routes, and fans out events to designated targets based on pre-defined routing rules.
- **Compute Worker (AWS Lambda-1):** A serverless compute function triggered by EventBridge. It cleanses, validates, and transforms raw incoming payloads into schema-compliant JSON documents.
- **Storage Layer (Amazon DynamoDB):** The transformed data is written into a NoSQL database. It acts as the high-throughput, low-latency persistent data store for the application.

#### 2. Downstream Streaming & Omni-Channel Execution (Steps 5, 15, 16)

This pathway responds to changes in the database to drive customer support center actions.

- **Change Data Capture (DynamoDB Streams):** Captures time-ordered modifications (inserts/updates) made to items inside the database tables in real time.
- **Stream Processor (AWS Lambda-4):** Consumes the raw stream shards from *DynamoDB*, filtering for specific status flags or conditions that require customer contact.
- **Contact Center Orchestration (Amazon Connect):** Invoked by the *Lambda* function to automatically spin up omni-channel customer service routines, outbound calls, or automated voice workflows.

### 3. Operational Outbound Reconciliation Path (Steps 6–9)

This pathway closes the feedback loop by returning execution results to *Salesforce*.

- **Change Detection Loop:** Changes from *DynamoDB Streams* also invoke *AWS Lambda-2* to format and store structural update events.
- **Staging Area (Amazon S3):** Object storage used to stage bulk transaction status files and process outputs before transferring them out of the *AWS* network.
- **Scheduled Synchronization (Amazon AppFlow-2):** Runs on a schedule-based trigger to pull processed files from *Amazon S3* and write the final event requests and processing statuses back into *Salesforce*.

### 4. Resiliency & Operational Alerting Framework (Steps 12–15)

This parallel framework isolates system failures and alerts internal teams without blocking the active data pipelines.

- **Fault-Tolerant Ingress Monitoring:** If *Amazon AppFlow-1* breaks or a payload fails ingestion, a specialized dead-letter rule in *Amazon EventBridge* instantly intercepts the failure.
- **Infrastructure Failure Triage (AWS Lambda-5 & SNS):** Processes the failure schema and sends it to *Amazon Simple Notification Service* (SNS). This dispatches urgent email notifications directly to the *AWS* Solution Support Team.
- **Database State Alerting (AWS Lambda-3 & SNS):** Monitors operational trends out of *DynamoDB Streams*. If an error is detected downstream, it sends high-level summary emails to Business Users & Support Teams.

# Comprehensive Solution Summary

## Comprehensive Solution Summary

The proposed solution delivers a highly resilient, serverless, and bi-directional integration engine that bridges *Salesforce* CRM with cloud-native *AWS* resources. By transitioning from traditional point-to-point connections to an asynchronous, event-driven pattern, the architecture guarantees real-time event processing, zero data loss, automated failure mitigation, and scalable omni-channel customer support orchestration.

### 1. End-to-End Managed Data Ingestion & Extraction Ingress Framework

The platform's data entry boundary decouples and aggregates event traffic arrays traveling between the core *SaaS* customer relationship management environment and internal *AWS* infrastructure.

- **Managed Event-Driven Ingestion Boundary:** Public-facing *CRM* actions trigger inbound event payloads that route directly into *Amazon AppFlow-1* on an event-driven basis. This layer abstracts *API* infrastructure management, absorbs sudden traffic spikes, and eliminates the heavy overhead of traditional point-to-point *HTTP* polling.
- **Asynchronous Enterprise Event Distribution:** Ingested payloads pass immediately into *Amazon EventBridge*, which serves as a highly scalable, decoupled central messaging bus. *EventBridge* applies strict pattern-matching and filtering rules to evaluate, decouple, and securely route the incoming event request streams without imposing downstream system dependencies.
- **High-Throughput Scheduled Synchronization Egress:** Outbound processing metrics and transaction closures aggregate inside a dedicated *Amazon S3* staging layer. A secondary, schedule-based instance of *Amazon AppFlow-2* routinely extracts these staged update batches, securely validating and writing execution states back to *Salesforce* as structured status outputs.

### 2. Multi-Tier Serverless Compute, Integration Routing, and Core Storage Ecosystem

The core computing ecosystem utilizes a completely serverless, stateless runtime topology to process high-velocity application logic, maintain database consistency, and track application state transitions.

- **Stateless Event-Driven Execution Layer:** Core transaction processing logic runs inside a fully managed runtime environment driven by *AWS Lambda-1*.

This function triggers instantly upon receiving event streams from *AWS EventBridge*, parsing raw event payloads, validating data integrity, and executing schema transformations on demand.

- **High-Performance Low-Latency State Persistence:** Validated transactional entities map directly into *Amazon DynamoDB*, a fully managed NoSQL database engine. *DynamoDB* securely isolates active operational data tables, delivering predictable, single-digit millisecond latency for heavy read and write workloads at any scale.
- **Real-Time Change Data Capture Stream Topology:** Database mutations, modifications, and inserts generate time-ordered transaction logs captured by *DynamoDB Streams*. This architecture completely decouples database commits from downstream consumption, fanning out change notifications simultaneously to multiple processing pathways without adding performance overhead to the database.
- **Centralized Governance, Failure Isolation, and Compliance Alerting Framework:** This resilient architecture deploys an absolute fault-tolerant monitoring paradigm across all ingestion paths and runtime layers to enforce deep pipeline visibility, real-time threat isolation, and proactive infrastructure alerting.
- **Automated Boundary Break Detection & Isolation:** The ingestion pipeline implements automated boundary protection rules; if the data connection breaks or standard ingestion thresholds fail, a specialized fault-tolerance path redirects the active payload straight into a dedicated error-handling *Amazon EventBridge* rule.
- **Infrastructure Triage and Tier-1 Support Notification:** Errant ingestion traces and dead-letter payloads trigger an isolated handler function via *AWS Lambda-5*. This function formats the error stack and routes it through *Amazon Simple Notification Service (SNS)*, dispatching urgent emails to the *AWS Solution Support Team* for immediate manual or automated remediation.
- **Operational Telemetry and Functional Exception Analytics:** Downstream operational trends coming from the streaming database layer route through *AWS Lambda-3* to detect systemic processing failures or data irregularities. Verified operational anomalies fan out via *Amazon SNS* to deliver high-level alert summaries directly to the Business Users & Support Team inbox.



### 3. Hybrid Network Topography, Stream Consumers, and High-Availability Execution Fabric

The operational infrastructure is engineered to trigger secondary, external business workflows and customer interactions seamlessly while ensuring continuous, low-latency transaction routing across cloud boundaries.

- **Dynamic Database Stream Consumption & Formatting:** Updates captured via change data capture trigger an independent background processing loop managed by *AWS Lambda-2*. This function serializes real-time operational deltas and logs them as immutable audit files within *Amazon S3* storage buckets.
- **Automated Omni-Channel Contact Center Orchestration:** Operational status mutations committed within the core database tables invoke *AWS Lambda-4* in real time. This worker function processes the contextual state update and programmatically triggers outbound workflows inside *Amazon Connect*.
- **Proactive Customer Experience Execution Fabric:** *Amazon Connect* receives the serverless trigger parameters to initiate real-time customer touchpoints, voice responses, or support agent routing loops. This guarantees that internal customer support channels match active cloud database states instantly without human intervention.

## End-End Customer Journey

### Step 1: Managed Event Ingestion and Push-Based CRM Triggering

- **The User Action:** A sales representative updates an account status to "Closed-Won" or modifies a high-value opportunity field inside the *Salesforce* CRM interface.
- **Salesforce Event Streaming:** Instead of relying on slow, batch-based API polling, *Salesforce* instantly publishes a real-time event notification. This payload contains the record ID, operational delta change strings, and core contextual business data.
- **Amazon AppFlow Ingestion:** The outbound event hits the entry boundary of *Amazon AppFlow-1*. *AppFlow* maintains a secure, managed connection with *Salesforce*, pulling the high-volume payload instantly over an encrypted channel while protecting *Salesforce* API governance limits.

## Step 2: Bus Architecture Pattern Matching and Asynchronous Routing

- **Amazon EventBridge Ingress:** *Amazon AppFlow-1* directly hands off the raw inbound *JSON* payload to *Amazon EventBridge*, the central serverless event broker.
- **Rule Evaluation & Filtering:** *EventBridge* evaluates the event headers and metadata against predefined schema rules. It isolates and matches the specific event type to prevent irrelevant data from cluttering downstream systems.
- **Decoupled Stream Distribution:** Once matched, *EventBridge* structurally decouples the event from the *Salesforce* origin. It instantly forwards the payload to its designated target—*AWS Lambda-1*—while maintaining a parallel fault-tolerance tracking loop.

## Step 3: Compute Transformation and High-Performance NoSQL Persistence

- **AWS Lambda Validation:** *AWS Lambda-1* launches an isolated, on-demand compute runtime to handle the event. It parses the inbound payload, checks for data completeness, and transforms the raw fields into a standardized *cloud-native JSON* schema.
- **Amazon DynamoDB Commit:** The transformed *JSON* document is written directly into *Amazon DynamoDB*. Working as a high-throughput, low-latency *NoSQL* database, *DynamoDB* commits the new transactional state in single-digit milliseconds.
- **Change Data Capture Activation:** The instant the database write finishes, *DynamoDB Streams* captures the exact modification. It creates an immutable, time-ordered log of the insert or update, instantly fanning out change notifications to downstream workers.

## Step 4: Outbound Event Staging and Audit Log Compilation

- **AWS Lambda Stream Processing:** The database mutation log in *DynamoDB Streams* automatically invokes *AWS Lambda-2*. This function extracts the operational delta, serializes the change parameters, and formats the data for outbound sync staging.
- **Amazon S3 Object Storage:** *AWS Lambda-2* writes the compiled operational status files directly into a private *Amazon S3* bucket. The *S3* bucket serves as an immutable, highly durable storage layer, holding the transaction logs securely until the next scheduled transfer window opens.

- **Data Verification:** The file write in S3 locks the updated transaction state safely inside the AWS storage perimeter, preparing it for bulk synchronization back to the CRM system.

#### Step 5: Scheduled CRM Reconciliation and Feedback Synchronization

- **Amazon AppFlow Triggering:** *Amazon AppFlow-2* executes based on a predefined chronological schedule or an automated time-interval trigger.
- **Bulk Data Extraction:** Upon activation, *AppFlow* connects to the designated *Amazon S3* bucket and extracts the staged transaction processing logs and final closure statuses.
- **Salesforce Record Closure:** *AppFlow* maps the data back into the original CRM structures and pushes the updates to *Salesforce*. The system updates the *Salesforce* records with the final AWS processing metrics, closing the bi-directional feedback loop and ensuring data consistency.

#### Step 6: Database Change Interception and Contact Center Invocation

- **Parallel Stream Extraction:** Simultaneously, the same database change log in *DynamoDB Streams* triggers a separate execution path via *AWS Lambda-4*, isolating markers that require instant human intervention.
- **Contextual State Evaluation:** *AWS Lambda-4* reviews the status flags (such as customer escalation codes or high-priority onboarding flags) to confirm if an automated phone call or omni-channel routing routine is needed.
- **Amazon Connect Triggering:** The function packages the customer's profile parameters and programmatically executes a real-time workflow request inside *Amazon Connect* via a *serverless* API call.

#### Step 7: Automated Omni-Channel Call Routing and Customer Engagement

- **Amazon Connect Execution:** *Amazon Connect* receives the serverless parameter packet and immediately initiates the targeted customer interaction workflow.
- **Outbound Customer Contact:** The system runs an automated outbound voice response or places the customer into a priority interactive voice response (IVR) queue.
- **Support Agent Sync:** The moment a live customer service representative answers the call, Connect injects the real-time *AWS database* context onto their desktop application, giving the agent a complete view of the transaction state without delay.

## Step 8: Fault Isolation, Triage, and Operational Alerting

- **Boundary Error Interception:** If *Amazon AppFlow-1* breaks or a payload fails schema validation, a specialized dead-letter pattern rule inside *Amazon EventBridge* catches the failure instantly to prevent data loss.
- **AWS Lambda Triage:** The failed event logs are routed to *AWS Lambda-5* (for ingress failures) or *AWS Lambda-3* (for downstream streaming failures). These functions parse the stack traces to isolate the exact cause of the breakdown.
- **Amazon SNS Notification:** The triage *Lambdas* send formatted exception alerts through *Amazon Simple Notification Service* (SNS). SNS splits the alerts by severity, sending critical infrastructure failures directly to the AWS Solution Support Team and high-level operational trends straight to the Business Users & Support Team inbox.

## Transaction Data-Flows

This section maps the structural journey into discrete, execution-based data flows. It separates real-time, synchronous transaction processing from non-blocking background logic and persistent analytical data mirroring..

### 1. Main Data Ingestion and Processing Flow

- **Salesforce Event Trigger:** Occurs when a platform event, *change data capture* (CDC), or outbound message is published. This is a non-blocking asynchronous push using *Salesforce's* streaming API architecture.
- **Amazon AppFlow-1 (Ingestion):** Operates in near-real-time as a fully managed integration service. It maps incoming *Salesforce JSON* fields to AWS native objects, handling schema translation and data type formatting automatically.
- **Amazon EventBridge (Routing):** Acts as the central event bus. It applies rule-based filtering (e.g., matching on event type or status) to match the incoming *JSON* payload and route it target-side to the *Lambda* function.
- **AWS Lambda-1 (Compute):** Receives the *JSON* payload event batch, unpacks it, applies validation/enrichment rules, and calls the *DynamoDB* PutItem or UpdateItem API.
- **Amazon DynamoDB & Streams:** The database receives the record. Upon a successful commit, a time-ordered sequence of item-level modifications (insert, update, delete) is written to the stream in near-real-time with a strict 24-hour retention window.

## 2. Status Feedback & Business Notification Flow

- **DynamoDB Streams to Lambda-2:** *Lambda-2* polls the shard inside the stream. It processes records in micro-batches (e.g., up to 1,000 records at a time) to ensure high throughput and minimize execution costs.
- **Amazon S3 (Staging):** *Lambda-2* writes the processed status changes as compressed batch files (typically .csv or .json formats) into a structured S3 prefix (e.g., s3://bucket/year/month/day/status-updates/).
- **Amazon AppFlow-2 (Egress):** Runs on a cron-based schedule (e.g., every 15 minutes, hourly, or daily). It scans the designated S3 prefix, picks up the status files, and updates *Salesforce* records using bulk APIs to respect *Salesforce* API governor limits.
- **Notification Loop (Lambda-3 & SNS):** *Lambda-3* concurrently reads the stream to filter for specific high-priority business events. It publishes an unformatted text or rich *HTML* payload to an *SNS* Topic, which fans out to email endpoints.

## 3. Contact Center Integration Flow

- **Event Filtering:** AWS *Lambda-4* is mapped to the *DynamoDB* Stream but ignores routine technical updates. It targets specific lifecycle changes, such as a customer case creation or VIP status flag change.
- **Amazon Connect API Integration:** *Lambda-4* translates the database update into contact center attributes. It interacts with *Amazon Connect* via the *StartOutboundVoiceContact* or *UpdateContactAttributes* APIs. This updates active agent screens or instantly adjusts interactive voice response (*IVR*) routing rules.

## 4. Error Handling & Fault Tolerance Flow

- **AppFlow Fallback:** If *AppFlow-1* encounters network timeouts, authentication issues, or payload validation errors, it raises an execution failure event.
- **EventBridge Error Rule:** A dedicated rule on the default event bus filters for *AppFlow* execution state changes where the status equals *FAILED*.
- **AWS Lambda-5 (Triage):** Receives the error context payload. It extracts crucial metadata including the failed execution ID, error code, timestamp, and target records.
- **Amazon SNS (Critical Alerting):** *Lambda-5* sends a high-priority payload to an administrative *SNS* topic. This topic formats the *JSON* into an

actionable email alert containing links to *AWS CloudWatch* logs for immediate troubleshooting.

## Security Architecture

This section outlines the defense-in-depth framework designed to protect customer data as it moves between the cloud and the salesforce.

### 1. Ingestion and Perimeter Security

- **Salesforce Connected Apps**

Authenticates *AppFlow* connections securely using the *OAuth 2.0* protocol with *JWT* bearer token flows. This architecture eliminates the need to store long-lived Salesforce usernames or passwords within the AWS ecosystem.

- **AWS PrivateLink Support**

Routes data directly over the *AWS private network backbone* instead of routing traffic across the public internet. This significantly reduces the attack surface and protects the event payloads from external interceptors or *DDoS* attacks.

- **EventBridge Bus Policies**

Enforces resource-based policies on the default *Amazon EventBridge* event bus to accept payloads only from designated *AppFlow* streams. Any unauthorized event sources attempting to publish to the bus are rejected immediately.

- **Ingestion IAM Roles**

Implements strict execution roles for *AWS Lambda-1* using the principle of least privilege. The function is given explicit permissions solely for `dynamodb:PutItem` and `dynamodb:UpdateItem` on the target table.

### 2. Processing and Storage Security

- **KMS Managed Encryption**

Secures the core *Amazon DynamoDB* database at rest using *AWS Key Management Service (KMS)* with *Customer Managed Keys (CMKs)*. This ensures complete governance over key rotation policies and provides a cryptographic audit trail.

- **Fine-Grained Stream IAM**

Restricts access to *DynamoDB* Streams by forcing *AWS Lambda-2* and *Lambda-3* to assume dedicated roles. These roles grant execution rights

strictly to specific *Amazon Resource Names* (ARNs) belonging to the shard paths.

- **VPC Endpoint Routing**

Deploys *Lambda* functions inside private subnets of a *Virtual Private Cloud* (VPC) utilizing interface *VPC endpoints*. This configuration keeps all inter-service compute operations entirely isolated from external networks.

- **Secrets Manager Integration**

Uses *AWS Secrets Manager* to inject external *API* tokens or configuration values securely into runtime execution environments. This workflow avoids the high-risk practice of hardcoding environment variables within the *Lambda* functions.

### 3. Feedback and Integration Security

- **S3 Block Public Access**

Enforces explicit "Block Public Access" configurations alongside strict bucket policies on the staging *Amazon S3* instance. This prevents any accidental exposure of the outgoing transactional status files to unauthorized public entities.

- **Minimal Egress Scopes**

Limits *Amazon AppFlow-2* configurations to minimal write privileges on the destination *Salesforce* instance. The connected *OAuth* scope is restricted to modify only the specified custom status objects, preventing unauthorized broad system access.

- **Secure Connect Attributes**

Uses *Amazon Connect* built-in security features to safeguard customer profile records and interaction data. Sensitive variables passed by *AWS Lambda-4* are encrypted in transit and mapped using locked system attributes within the contact flow.

- **Restricted SNS Publishing**

Applies strict resource policies to the *Amazon SNS* notification topics to allow only *SNS:Publish* actions from *Lambda-3*. This structure blocks malicious actors from spoofing or injecting fake business notifications into the email stream.

### 4. Governance and Error Security

- **CloudWatch PII Masking**

Implements data protection policies in *Amazon CloudWatch* Logs to automatically detect and mask *Personally Identifiable Information* (PII). This prevents sensitive *Salesforce* customer fields from appearing in plain text across application logs.

- **CloudTrail Control Logging**

Monitors and logs every API transaction across all components using *AWS CloudTrail*. This provides the compliance team with immutable, time-stamped logs of exactly who accessed or modified the system infrastructure.

- **Encrypted Dead-Letter Queues**

Routes any failed *AppFlow* executions or un-routable *EventBridge* payloads directly into an encrypted *Amazon SQS* Dead-Letter Queue (DLQ). This mechanism guarantees that broken transactions are isolated safely for debugging without causing data loss.

- **SNS Server-Side Encryption**

Activates server-side encryption (SSE) on the administrative *SNS* alert topics using *AWS KMS* keys. This provides a baseline layer of data protection for internal error payloads before they are delivered to the support team.

## Disaster Recovery (DR) Plan

This plan ensures business continuity, system availability, and zero data corruption if an infrastructure failure or natural disaster takes down a cloud availability zone or the on-premises data centre.

### 1. Multi-Region Replication and Infrastructure Strategy

- **DynamoDB Global Tables**

Enables fully managed, active-active cross-region replication to a secondary *AWS* region with a latency of under one second. This configuration ensures that if the primary region goes offline, an identical copy of the transactional database is instantly available for compute services.

- **S3 Cross-Region Replication (CRR)**

Automates the asynchronous replication of staged status update files from the primary *Amazon S3* bucket to a target bucket in the disaster recovery region. *S3 Versioning* is enabled on both buckets to prevent data loss from accidental deletion, overwrites, or file corruption.

- **Infrastructure as Code (IaC) Parity**

Maintains synchronized deployments of all *AWS Lambda* functions, *EventBridge* rules, and *Amazon SNS* topics across both regions using



*Terraform* or *AWS CloudFormation*. This guarantees configuration consistency and eliminates drift, ensuring the secondary region is always ready to accept live traffic.

- **Pre-Configured AppFlow Standby Bundles**

Deploys inactive, identical instances of *Amazon AppFlow-1* and *AppFlow-2* configurations inside the secondary region. These connections point to the same *Salesforce* environment but remain dormant until a failover event occurs, avoiding duplicate processing or governor limit issues.

## 2. Monitoring, Health Checks, and Failover Automation

- **Route 53 Application Recovery Controller (ARC)**

Monitors the end-to-end health of the primary ingestion pipeline using synthetic canaries and regional *CloudWatch* metrics. *ARC* manages routing control states, enabling automated or single-click routing redirection during high-severity service disruptions.

- **EventBridge Global Endpoints**

Utilizes built-in *Amazon EventBridge* global routing capabilities to automatically failover ingest streams if the primary bus becomes unreachable. This architecture safely buffers incoming payloads on the AWS edge network while transitioning compute workloads to the backup region.

- **CloudWatch Cross-Region Alarms**

Aggregates error metrics, *Lambda* throttling events, and *AppFlow* execution failures into a centralized operational dashboard. When specified threshold limits are breached consecutively for 3 minutes, it triggers an automated failover sequence via *AWS Systems Manager*.

- **Automated Step Functions Runbook**

Executes a coordinated failover orchestration script via *AWS Step Functions* when an outage is declared. This runbook activates the secondary *AppFlow* instances, updates *DNS* routing records, and flips the execution state of the backup *Lambda* event source mappings.

## 3. Salesforce-Side Rerouting and Data Recovery

- **Salesforce Named Credential Updates**

Modifies the endpoint parameters within *Salesforce* Named Credentials dynamically using an apex call or automated flow during a disaster scenario. This redirects the outbound *Event Request* Payload stream away from the failed primary region to the healthy secondary *AWS endpoint*.

- **Event Replay ID Utilization**

Leverages *Salesforce*'s 72-hour event retention window by utilizing Event Replay IDs if *AppFlow*-1 suffers ingestion gaps during the failover window. The secondary *AppFlow* instance requests past event streams starting from the exact last-known successful replay ID to achieve zero data loss.

- **Bulk API Failover Strategy**

Switches the scheduled status feedback loops (*AppFlow*-2) to use *Salesforce Bulk API* 2.0 when recovering from long outages. This optimizes processing efficiency and handles large backlogs of staged *S3* status updates without hitting daily transaction limits.

- **Bidirectional Correlation Tracking**

Embeds a unique, immutable *UUID* tracking ID within the *Salesforce* Event Request Payload that stays attached to the data across *DynamoDB* and *S3*. This shared reference allows automated reconciliation scripts to quickly identify dropped messages between the two platforms during a regional flip.

#### **4. Failback, Data Reconciliation, and Testing**

- **Regional Parity and Delta Validation**

Verifies that the primary region is fully stable and that all data deltas have completed replication before initiating a failback. The operations team reviews global table replication lag metrics to confirm that both regional databases are in absolute sync.

- **Traffic Draining Phase**

Allows active transactions inside the secondary region's *SQS* Dead-Letter Queues and *DynamoDB* Streams to process completely before shifting traffic back. This controlled cooldown period ensures that no mid-flight operations are cut off or corrupted during the transition.

- **Phased Traffic Shifting**

Employs an incremental canary routing approach using *Route 53* to shift 10% of the ingestion traffic back to the primary region first. System performance and error rates are monitored for a 15-minute soak period before routing the remaining 90% of production traffic.

- **Chaos Engineering and Game Days**

Mandates quarterly, non-disruptive disaster recovery testing using *AWS Fault Injection Service* (FIS) to simulate regional outages in a staging environment. These simulations validate that automated recovery patterns perform successfully within the targeted 10-minute recovery window.

# Architecture Tradeoffs

## 1. Ingestion Strategy: Event-Driven Push via Amazon AppFlow

- **The Benefit**

Provides near-real-time data ingestion with minimal operational overhead, eliminating the need to build, manage, or scale a custom webhook listener infrastructure.

- **The Trade-off**

Exposes the system directly to *Salesforce* API governor limits and transactional bursts, which can exhaust daily integration allocations during peak traffic volumes.

- **The Mitigation**

Implement *EventBridge* rate-limiting rules and introduce an *Amazon SQS* buffer directly after *AppFlow* to smooth out sudden, volatile traffic spikes.

## 2. Storage Strategy: NoSQL via Amazon DynamoDB & Streams

- **The Benefit**

Delivers predictable, single-digit millisecond performance at any scale while automatically capturing all item-level data mutations chronologically via integrated change streams.

- **The Trade-off**

Restricts complex relational queries, ad-hoc reporting, and multi-table joins, forcing application code to handle data relationships that are native to SQL databases.

- **The Mitigation**

Design strict *Global Secondary Indexes (GSIs)* based on application query patterns, or pipe stream data into *Amazon Athena* for ad-hoc analytical reporting.

## 3. Feedback Loop: Schedule-Based Batch Processing to Salesforce

- **The Benefit**

Protects *Salesforce* from being overwhelmed by continuous transactional updates, maximizing API efficiency by bundling status records into controlled, bulk uploads.

- **The Trade-off**

Introduces a deterministic processing delay (data latency) into the business feedback loop, meaning users see stale transaction statuses until the next schedule runs.

- **The Mitigation**

Implement a dual-lane routing logic that uses immediate real-time platform events for critical customer failures while keeping routine updates on the cheaper batch schedule.

#### **4. Availability Strategy: Multi-Region Active-Active Warm Standby**

- **The Benefit**

Guarantees near-zero data loss (*RPO* < 1s) and ultra-low downtime (*RTO* < 10m) by keeping an identical, live backup infrastructure running in a secondary AWS region.

- **The Trade-off**

Doubles baseline infrastructure operational costs and introduces complex data split-brain or write-conflict resolution risks across the regional database nodes.

- **The Mitigation**

Utilize *DynamoDB Global Tables* built-in "Last-Writer-Wins" automatic conflict resolution rule, and deploy *Route 53 Application Recovery Controller* to safely manage regional routing states.

#### **5. Integration Scope: Tight Coupling via Native AWS/Salesforce Adapters**

- **The Benefit**

Accelerates time-to-market and reduces custom coding by using out-of-the-box *Amazon AppFlow* connectors that handle authentication, retries, and data transformations automatically.

- **The Trade-off**

- Creates deep architectural lock-in to specific cloud provider ecosystems, making it highly difficult and costly to migrate components to other integration platforms or cloud vendors later.

- **The Mitigation**

Encapsulate downstream business logic within standardized, reusable AWS *Lambda* microservices, ensuring that the core data processing layer remains independent of the ingestion adapter.

# Architecture Decision Records

## 1. Ingestion Engine Selection

- **Decision**  
Use *Amazon AppFlow* instead of building a custom *API Gateway* and *AWS Lambda webhook listener*.
- **Rationale**  
*AppFlow* natively manages *Salesforce OAuth* token refreshes, handles API retries, and automatically scales with traffic spikes. This minimizes custom infrastructure maintenance and allows development teams to focus purely on business logic rather than integration plumbing.

## 2. Primary Database Selection

- **Decision**  
Utilize *Amazon DynamoDB* as the core transactional database instead of a relational option like *Amazon RDS*.
- **Rationale**  
The workload requires predictable, single-digit millisecond write latencies to keep pace with rapid *Salesforce* streaming events. *DynamoDB* handles high-throughput operational bursts effortlessly and features native change data capture (CDC) through *DynamoDB Streams*.

## 3. Change Data Capture Mechanism

- **Decision**  
Implement *DynamoDB Streams* to drive downstream feedback loops, alerts, and contact center updates.
- **Rationale**  
This architecture decouples the primary ingestion path from downstream operations. By processing mutations asynchronously from the stream, the main event processing flow remains highly performant and unaffected by failures or slowdowns in reporting or alerting sub-systems.

## 4. Downstream Egress Routing to Salesforce

- **Decision**  
Use schedule-based batch execution via *Amazon AppFlow-2* staged in *Amazon S3* for routine status updates.
- **Rationale**  
Updating *Salesforce* immediately for every single transaction would

rapidly exhaust daily *Salesforce* API governor limits. Staging updates in compressed *S3* files and pushing them in bulk batches optimizes API consumption and significantly lowers integration costs.

## 5. Multi-Region High Availability Strategy

- **Decision**

Deploy a *multi-region Warm Standby* architecture utilizing *DynamoDB Global Tables* and *Route 53 Application Recovery Controller (ARC)*.

- **Rationale**

The business requires a stringent *Recovery Time Objective (RTO)* of under 10 minutes and a *Recovery Point Objective (RPO)* of under 1 second. Active-Active database replication paired with automated DNS-level routing handles regional *AWS* outages seamlessly without risking data loss.

## 6. Event Routing Architecture

- **Decision**

Route incoming *AppFlow* event payloads through *Amazon EventBridge* before triggering compute functions.

- **Rationale**

*EventBridge* acts as a central event bus that decouples ingestion from execution. Its native rule-based filtering allows the system to easily evaluate payload attributes and direct messages to specific *Lambda* targets without hardcoding logic into the compute layer.

## 7. Configuration and Credential Management

- **Decision**

Store all integration tokens, *API* keys, and sensitive environment configurations inside *AWS Secrets Manager*.

- **Rationale**

Hardcoding configurations or using plain-text *Lambda* environment variables exposes the system to severe security risks. *Secrets Manager* secures data at rest, audits access automatically via *CloudTrail*, and enables seamless operational secret rotation.

## 8. Compute Layer Design Pattern

- **Decision**

Utilize small, single-purpose *AWS Lambda* functions (Microservices) rather than a large monolithic function.

- **Rationale**

Splitting workloads into dedicated tasks (e.g., Ingestion, Feedback, Contact Center, Errors) simplifies code maintenance and establishes strict security boundaries. Each function can be granted the exact least-privilege *IAM* permissions required for its specific operational role.

## 9. Operational Error and Exception Handling

- **Decision**

Implement an encrypted *Amazon SQS* Dead-Letter Queue (*DLQ*) attached to both *EventBridge* and *Lambda* targets.

- **Rationale**

Malformed payloads or unexpected downstream timeouts can cause transient processing failures. A *DLQ* ensures that un-routable or broken transactions are safely isolated in an encrypted storage state for forensic debugging, completely eliminating silent data drops.

## 10. Observability and PII Protection

- **Decision**

Implement *CloudWatch* Logs Data Protection policies to automatically detect and mask sensitive customer fields.

- **Rationale**

Debugging transactional issues often requires looking at system application logs. Masking *Personally Identifiable Information* (PII) at the log ingestion layer ensures compliance with strict data privacy mandates (like *GDPR* or *CCPA*) without hindering engineering support teams.

# AWS Well-Architected Framework Review

An evaluation of this architecture against the design principles of the AWS Well-Architected Framework reveals the following target alignments and gaps:

## 1. Security Pillar

- **Strengths** Using *Amazon AppFlow* completely removes the risk of storing long-lived *Salesforce* credentials on *AWS*, leveraging secure *OAuth 2.0 JWT*

bearer flows instead. Relying on *VPC Interface Endpoints* and *PrivateLink* keeps ingestion and execution compute entirely off the public internet, neutralizing external interception vectors. *CloudWatch* Data Protection policies proactively mask *Personally Identifiable Information* (PII) at the ingestion boundary before logs hit persistent storage.

- **Recommendations:** Move from standard *RBAC* to *Attribute-Based Access Control* (ABAC) on your *DynamoDB* Streams to dynamically restrict access based on classification tags (e.g., *DataSensitivity: High*). Enable automatic 30-day rotation schedules in *AWS Secrets Manager* for any third-party tokens or secondary API keys, using an automated *Lambda* rotator to verify connectivity.

## 2. Reliability Pillar

- **Strengths:** Utilizing *DynamoDB* Global Tables provides seamless, active-active multi-region data replication, securing a target *Recovery Point Objective* (RPO) of under 1 second. *Route 53 Application Recovery Controller* combined with *EventBridge* Global Endpoints ensures near-instantaneous infrastructure routing shifts during regional *AWS* disruptions. Leveraging *Salesforce*'s 72-hour event retention window via Event Replay IDs guarantees that missing data blocks can be pulled and reprocessed upon pipeline recovery
- **Recommendations:** Implement a circuit breaker pattern inside your *Lambda* function retry logic when communicating with external endpoints like *Amazon Connect* to avoid cascading function timeouts. Ensure that your core ingestion function (*Lambda-1*) utilizes a unique identifier (like a *Salesforce* Event UUID) as a *DynamoDB* idempotency key to safely handle duplicate message deliveries gracefully.

## 3. Cost Optimization Pillar

- **Strengths:** Staging status update files in *Amazon S3* and using a scheduled *AppFlow* cron prevents expensive *Salesforce* API overage charges by taking advantage of efficient Bulk APIs. Operating on a purely serverless stack (*AppFlow*, *EventBridge*, *Lambda*, *DynamoDB*) means your baseline idle cost is effectively zero when *Salesforce* is not sending events. Using native *SaaS* connectors like *AppFlow* drastically lowers custom development, testing, and continuous engineering upkeep costs.



- **Recommendations:** Establish strict *S3 Lifecycle* policies to automatically transition staging status files to *S3 Glacier Instant Retrieval* after 7 days, and permanently delete them after 30 days. Transition *DynamoDB* from On-Demand capacity to *Provisioned Capacity* with *Auto Scaling* configured once production traffic baselines stabilize to reduce raw database spend by up to 50-70%.

#### 4. Performance Efficiency Pillar

- **Strengths:** Offloading data persistence to *Amazon DynamoDB* ensures predictable, single-digit millisecond performance even during heavy seasonal transactional spikes from *Salesforce*. Using *DynamoDB Streams* ensures that heavy secondary processes (like triggering *Amazon Connect* or building status files) do not introduce latency into the core write path. Employing independent, single-purpose *AWS Lambda* functions allows each *microservice* to scale and allocate memory precisely according to its execution profile.
- **Recommendations:** Adjust *Lambda* stream event source mappings by modifying `BatchSize` and `MaximumBatchingWindowInSeconds` to maximize micro-batch density and reduce overall *Lambda* execution counts. Apply *Provisioned Concurrency* to *Lambda-1* (the main ingestion function) to completely eliminate cold-start latencies during sudden outbound transaction bursts from *Salesforce*.

#### 5. Operational Excellence Pillar

- **Strengths:** Deploying identical resources across primary and secondary regions via *Infrastructure as Code* (IaC) ensures configuration drift is structurally impossible. Attaching dedicated, encrypted *SQS Dead-Letter Queues* (DLQs) to each processing hop ensures that bad data payloads can be isolated without disrupting healthy pipelines. Having a dedicated error-handling routing bus via *EventBridge* instantly flags execution failures and generates actionable alerting templates for support engineers.
- **Recommendations:** Implement *AWS X-Ray* across all *Lambda* functions and *EventBridge* buses to achieve complete, end-to-end trace visibility of a transaction from *AppFlow* down to the *S3* bucket. Schedule biannual, non-disruptive infrastructure failure game days using *AWS Fault Injection Service* (FIS) to validate that your support team can seamlessly interpret error logging structures.

## 6. Sustainability Pillar

- **Strengths:** Relying completely on fully managed services implies high underlying hardware utilization, minimizing wasted physical server capacity. Combining small statuses into heavy batch updates rather than keeping continuous open connections drastically optimizes network transmission energy footprints.
- **Recommendations:** Migrate all *AWS Lambda* runtimes from the *x86\_64* architecture to *arm64* (*AWS Graviton2/Graviton3*). This adjustment delivers better price-performance while significantly lowering energy consumption per compute cycle. Review memory allocation limits on your status processing functions to ensure they complete their batch tasks efficiently without leaving idle compute memory unutilized.

## Architecture Risks

### 1. Ingestion Bottleneck Risk

- **The Flaw**

*Amazon AppFlow-1* streams incoming *Salesforce* platform events directly into Amazon EventBridge without an intermediary buffering tier.

- **The Risk**

Sudden bursts or massive spikes in *Salesforce* transactional volume can trigger downstream *AWS Lambda-1* throttling, causing dropped events and data loss if concurrent execution limits are breached.

- **The Fix**

Introduce an *Amazon SQS* FIFO queue immediately between *AppFlow-1* and *EventBridge* to act as a buffer, smoothing out traffic spikes and ensuring zero data drop.

### 2. Tight Adapter Lock-In Risk

- **The Flaw**

The architecture relies heavily on native, out-of-the-box *Amazon AppFlow* connectors to manage authentication, payload mapping, and direct data transport.

- **The Risk**

Migrating to a different middleware, cloud provider, or hybrid integration platform in the future will require a complete rewrite of the ingestion and transformation layer.

- **The Fix**

Encapsulate downstream data handling using clean architecture patterns inside *AWS Lambda-1*, separating the core business validation logic from the *AppFlow* event structure.

### **3. Missing Idempotency Processing Risk**

- **The Flaw**

*AWS Lambda-1* inserts incoming data into *Amazon DynamoDB* without verifying if the specific event ID has already been executed.

- **The Risk**

Network retries from *Salesforce* or *AppFlow* can deliver duplicate payloads, leading to overwritten records, distorted business data, and corrupted database integrity.

- **The Fix**

Configure *DynamoDB* conditional writes using a unique *Salesforce* Event UUID as a conditional key constraint to silently reject duplicate incoming payloads.

### **4. Direct External Call Slower Execution Risk**

- **The Flaw**

*AWS Lambda-4* directly calls external *Amazon Connect* APIs during synchronous stream execution without handling downstream connection slowdowns.

- **The Risk**

If the contact center API encounters high latency, the *Lambda* function will experience prolonged executions, driving up compute costs and causing stream processing blockages.

- **The Fix**

Apply a strict circuit breaker pattern using an intermediate *Amazon SQS* queue to decouple *Lambda-4* from the *Amazon Connect* execution boundary entirely.

### **5. Stale Business Feedback Window Risk**

- **The Flaw**

*Amazon AppFlow-2* relies entirely on a cron-based schedule to pull transaction status files from *Amazon S3* and update *Salesforce*.

- **The Risk**

Business operators will experience data latency and view stale transaction statuses in *Salesforce* between schedule runs, delaying customer service actions.

- **The Fix**

Implement a dual-lane strategy where high-priority failures trigger real-time *Salesforce* platform events, while routine updates remain on the cost-effective batch schedule.

## 6. Relational Query Constraints Risk

- **The Flaw**

The architecture utilizes *Amazon DynamoDB*, a *NoSQL* data store, as its sole repository for transactional event processing data.

- **The Risk**

Business teams will be unable to run complex ad-hoc relational queries, multi-table joins, or deep reporting directly against the operational database.

- **The Fix**

Stream data mutations out of *DynamoDB* via *Kinesis Data Firehose* into an *Amazon S3* data lake, allowing teams to query logs using *Amazon Athena*.

## 7. Salesforce API Limit Exhaustion Risk

- **The Flaw**

The return status pipeline updates *Salesforce* data without tracking or enforcing real-time governor limit metrics.

- **The Risk**

A sudden spike in transaction volume can completely consume daily *Salesforce* API allocations, breaking other critical enterprise integrations connected to the platform.

- **The Fix**

Configure *AppFlow-2* to use the *Salesforce* Bulk API 2.0 pattern and include a circuit breaker that pauses sync operations if remaining API limits drop below 10%.

## 8. Database Split-Brain and Write Conflict Risk

- **The Flaw**

The Disaster Recovery strategy relies on an active-active *DynamoDB* Global Tables configuration spread across two distinct *AWS regions*.

- **The Risk**

Concurrent regional updates to the exact same record can cause data discrepancies or race conditions across nodes if write order is not controlled.

- **The Fix**

Enforce *DynamoDB's* built-in "Last-Writer-Wins" deterministic conflict resolution rule and maintain application-level sequence tracking using a transaction timestamp.

## 9. Log Injection of Unmasked PII Risk

- **The Flaw**

Application runtime errors print the entire raw *Salesforce* event payload directly into *Amazon CloudWatch* Logs for debugging purposes.

- **The Risk**

*Personally Identifiable Information* (PII) will sit exposed in plain-text logs, violating compliance regulations like *GDPR*, *HIPAA*, or *CCPA*.

- **The Fix**

Deploy active *CloudWatch* Logs Data Protection policies to automatically detect, mask, and redact sensitive fields like emails, phones, and names at ingestion.

## 10. Lambda Cold Start Latency Risk

- **The Flaw**

The main ingestion compute engine (*AWS Lambda-1*) runs on standard, on-demand serverless allocation without pre-warmed execution environments.

- **The Risk**

Periods of low traffic followed by sudden *Salesforce* event bursts will trigger cold starts, causing unpredictable latency spikes for critical business transactions.

- **The Fix**

Configure Provisioned Concurrency on the ingestion *Lambda* function to ensure a dedicated baseline of pre-warmed environments is always active.

# Architectural Recommendations

## 1 . Ingestion Traffic Buffering

- **Action**

Introduce an *Amazon SQS* FIFO queue directly between *Amazon AppFlow-1* and *Amazon EventBridge*.

- **Recommendation**

This isolates the core processing pipeline from sudden transactional bursts in *Salesforce*, preventing downstream *AWS Lambda* throttling and ensuring zero data loss during high-volume spikes.

## 2. Payload Duplicate Prevention

- **Action**

Implement *DynamoDB* Conditional Writes inside the ingestion compute function (*AWS Lambda-1*).

- **Recommendation**

Enforce a strict application-level check using the unique *Salesforce* Event *UUID* as the condition key constraint to safely reject duplicate incoming messages and prevent data corruption.

## 3. Compute Latency Optimization

- **Action**

Configure Provisioned Concurrency on the primary ingestion function (*AWS Lambda-1*).

- **Recommendation**

Maintain a pre-warmed baseline of execution environments to eliminate cold-start latencies, guaranteeing predictable single-digit millisecond performance during sporadic traffic bursts.

## 4. Downstream API Decoupling

- **Action**

Isolate the contact center communication (*AWS Lambda-4*) using an asynchronous buffer queue.

- **Recommendation**

Decouple the *DynamoDB* Stream processing loop from direct *Amazon Connect* API calls to prevent external network latencies or timeouts from blocking the main data stream.

## 5. API Governor Limit Protection

- **Action**

Transition *Amazon AppFlow-2* to utilize *Salesforce* Bulk API 2.0 instead of standard *REST endpoints*.

- **Recommendation**

Group the outgoing staging status files from *Amazon S3* into dense, optimized batches to radically minimize daily *Salesforce* API consumption and prevent integration exhaustion.

## 6. Relational Reporting and Analytics

- **Action**

Deploy an *Amazon Kinesis Data Firehose* to stream *DynamoDB* data mutations directly into an *Amazon S3 data lake*.

- **Recommendation**

Enable business intelligence teams to run complex ad-hoc relational queries and multi-table joins using *Amazon Athena* without impacting the performance of the operational *NoSQL* database.

## 7. Storage Lifecycle Cost Reductions

- **Action**

Apply an automated *S3 Lifecycle Policy* to the staging status update bucket.

- **Recommendation**

Transition processed status tracking files to *S3 Glacier Instant Retrieval* after 7 days and configure permanent deletion after 30 days to systematically reduce storage spend.

## 8. Compute Architecture Modernization

- **Action**

Migrate all five *AWS Lambda* function runtimes from standard x86\_64 to the arm64 (*AWS Graviton*) processor architecture.

- **Recommendation**

Lower runtime energy footprints to meet corporate sustainability goals while instantly achieving up to 34% better price-performance per compute execution cycle.

## 9. Component Traceability and Observability

- **Action**

Activate *AWS X-Ray* distributed tracing across all *EventBridge* buses, *Lambda* microservices, and storage layers.

- **Recommendation**

Establish end-to-end transaction visibility with detailed service maps to let support engineers rapidly isolate bottlenecks and trace a single payload's lifecycle from *Salesforce* to *AWS*.

## 10. Automated Infrastructure Resilience Testing

- **Action**

Incorporate *AWS Fault Injection Service* (FIS) into the quarterly CI/CD deployment pipeline.

- **Recommendation**

Run automated chaos engineering game days that simulate regional network disconnects, validating that *Route 53 Application Recovery Controller* executes the 10-minute failover runbook flawlessly.

## Boundary Conditions for Reference Architecture

### 1 Ingestion Rate Saturation

- **The Issue**

*Amazon AppFlow-1* encounters transaction concurrency throttling when *Salesforce* pushes events at a rate that exceeds maximum regional API execution limits.

- **The Condition**

The outbound event burst rate exceeds 2,000 requests per second concurrently, or surpasses the default *AppFlow* concurrent execution quota for the regional *AWS* account.

### 2. Salesforce Daily API Exhaustion

- **The Issue**

The feedback status pipeline updates *Salesforce* record instances too frequently, completely consuming the target environment's daily integration allocation.

- **The Condition**

The combined volume of real-time events and schedule-based *AppFlow-2* updates exceeds the maximum *Salesforce* enterprise allocation of 100,000 *REST/Bulk API* calls within a rolling 24-hour window.



### 3. Payload Volume Ingestion Caps

- **The Issue**

Salesforce publishes a data modification event payload that contains heavily nested objects, exceeding native event bus message size limits.

- **The Condition**

The individual *JSON* event payload size exceeds exactly 256 KB, triggering an immediate rejection at the *Amazon EventBridge* ingestion boundary.

### 4. Cold Start Latency Spikes

- **The Issue**

*AWS Lambda-1* encounters severe cold start processing delays, causing upstream timing mismatches during erratic, low-frequency event cycles.

- **The Condition**

The system processes zero transactions for a continuous period of 15 minutes or longer, causing *AWS* to de-provision underlying microservice execution containers.

### 5. DynamoDB Single Partition Hotspotting

- **The Issue**

A high-volume transactional customer account causes unequal write distribution across the database, triggering partition-level performance throttling.

- **The Condition**

A single partition key (e.g., a specific Salesforce Account ID) experiences traffic that breaches the hard boundary of 1,000 writes per second or 3,000 reads per second.

## Appendix A: Non-Functional Requirements (NFR) Matrix

This matrix defines the critical quality attributes, operational constraints, and performance benchmarks that the system must satisfy to ensure long-term stability, security, and scalability.

| NFR Category                  | Target Objective                                                                                                                                                                      | Engineering Validation Mechanism(Tools)                                                                                                   | Architecture Component Mapping                       |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Performance & Latency         | <ul style="list-style-type: none"><li>End-to-end event processing payload delivery in &lt;2 seconds.</li><li>DynamoDB read/write latencies under 10 ms.</li></ul>                     | <ul style="list-style-type: none"><li>AWS X-Ray (Distributed Tracing)</li><li>Artillery / JMeter (Load Testing)</li></ul>                 | Amazon AppFlow<br>AWS Lambda<br>Amazon DynamoDB      |
| Scalability & Elasticity      | <ul style="list-style-type: none"><li>Handle sudden traffic spikes up to 5x baseline without manual intervention.</li><li>Zero provisioning overhead.</li></ul>                       | <ul style="list-style-type: none"><li>AWS Load Testing Framework</li><li>CloudWatch Metrics (Throttling, Concurrent Executions)</li></ul> | AWS Lambda<br>Amazon EventBridge<br>DynamoDB Streams |
| Reliability & Fault Tolerance | <ul style="list-style-type: none"><li>Zero data loss for broken flows (100% delivery guarantee via DLQs).</li><li>Automated alerts sent within 1) minute of a flow failure.</li></ul> | <ul style="list-style-type: none"><li>Chaos Engineering (AWS Fault Injection Service).</li><li>Failure injection testing.</li></ul>       | Amazon EventBridge, AWS Lambda, Amazon SNS           |
| Security & Compliance         | <ul style="list-style-type: none"><li>Data encrypted in transit (TLS 1.3) and at rest (AWS KMS Customer Managed Keys).</li><li>Least-privilege IAM roles.</li></ul>                   | <ul style="list-style-type: none"><li>AWS IAM Access Analyzer</li><li>AWS Security Hub</li><li>Salesforce Shield</li></ul>                | Amazon S3, Amazon DynamoDB, Salesforce Integration   |
| Observability & Alerting      | <ul style="list-style-type: none"><li>100% visibility into failed payloads.</li><li>Real-time notifications for</li></ul>                                                             | <ul style="list-style-type: none"><li>Amazon CloudWatch Logs &amp; Alarms</li><li>Datadog / New Relic</li></ul>                           | Amazon SNS, AWS Solution Support Team                |

|                                               |                                                                                                                                                                                                                        |                                                                                                                                                                      |                                                     |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
|                                               | critical system support events.                                                                                                                                                                                        |                                                                                                                                                                      |                                                     |
| <b>Availability &amp; Business Continuity</b> | <ul style="list-style-type: none"> <li>• Achieve 99.99% uptime for the integration pipeline.</li> <li>• Recovery Time Objective (RTO) &lt; 15 mins.</li> <li>• Recovery Point Objective (RPO) &lt; 1 min</li> </ul>    | <ul style="list-style-type: none"> <li>• AWS Resilience Hub</li> <li>• AWS Fault Injection Service (Simulated region failover)</li> </ul>                            | Amazon AppFlow, Amazon EventBridge, Amazon DynamoDB |
| <b>Data Retention &amp; Lifecycle</b>         | <ul style="list-style-type: none"> <li>• Archive S3 payloads after 90 days to Glacier.</li> <li>• Auto-purge ephemeral Lambda logs after 14 days.</li> <li>• Enforce DynamoDB TTL for temporary state data.</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Lifecycle Policies</li> <li>• AWS Config</li> <li>• Compliance Rules</li> <li>• CloudWatch Log Group Settings</li> </ul> | Amazon S3<br>Amazon DynamoDB<br>AWS Lambda (All)    |
| <b>API Governance &amp; Limits</b>            | <ul style="list-style-type: none"> <li>• Prevent integration from exhausting Salesforce API limits</li> <li>• Throttling triggers if Salesforce limit usage hits 80%.</li> </ul>                                       | <ul style="list-style-type: none"> <li>• Salesforce Event Monitoring</li> <li>• CloudWatch Alarms on AppFlow error codes</li> </ul>                                  | Salesforce Platform<br>Amazon AppFlow               |
| <b>Maintainability &amp; Testability</b>      | <ul style="list-style-type: none"> <li>• 100% of infrastructure deployed via code (IaC).</li> <li>• Lambda functions must maintain &gt;80% test coverage.</li> </ul>                                                   | <ul style="list-style-type: none"> <li>• Checkov / TFLint (IaC scanning)</li> <li>• SonarQube (Code quality pipeline)</li> </ul>                                     | Entire AWS Stack<br>AWS Lambda                      |
| <b>Auditability &amp; Traceability</b>        | <ul style="list-style-type: none"> <li>• Maintain an immutable audit log of all data mutations.</li> <li>• Retain compliance history for 7 years.</li> </ul>                                                           | <ul style="list-style-type: none"> <li>• AWS CloudTrail</li> <li>• AWS Artifact</li> <li>• DynamoDB Point-in-Time Recovery (PITR)</li> </ul>                         | Amazon DynamoDB<br>Amazon S3<br>CloudTrail Logs     |
| <b>Cost Efficiency</b>                        | <ul style="list-style-type: none"> <li>• Infrastructure cost must scale linearly with volume.</li> </ul>                                                                                                               | <ul style="list-style-type: none"> <li>• AWS Cost Explorer</li> <li>• AWS Compute Optimizer (Lambda memory tuning)</li> </ul>                                        | AWS Lambda (Serverless tier)                        |

|                                                |                                                                                                                                                                                                       |                                                                                                                                                                     |                                                         |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
|                                                | <ul style="list-style-type: none"> <li>• Max target cost of \$1.50 per 100,000 processed events.</li> </ul>                                                                                           |                                                                                                                                                                     | DynamoDB (On-Demand)                                    |
| <b>Idempotency &amp; Deduplication</b>         | <ul style="list-style-type: none"> <li>• Prevent duplicate record creation in AWS or Salesforce during network retries.</li> <li>• Zero duplicate payloads processed downstream.</li> </ul>           | <ul style="list-style-type: none"> <li>• Automated Integration Testing (Postman/Newman retry simulations)</li> <li>• DynamoDB unique constraint tracking</li> </ul> | AWS Lambda-1 & 2<br>Amazon DynamoDB (Idempotency Table) |
| <b>Data Sovereignty &amp; Residency</b>        | <ul style="list-style-type: none"> <li>• Ensure sensitive customer data never leaves designated geographical borders.</li> <li>• Restrict cross-border data replication.</li> </ul>                   | <ul style="list-style-type: none"> <li>• AWS Config Rules (Region restrictions)</li> <li>• Salesforce Data Masking policies</li> </ul>                              | Amazon S3<br>Amazon DynamoDB<br>Salesforce Org          |
| <b>Rate Limiting &amp; Concurrency Control</b> | <ul style="list-style-type: none"> <li>• Protect Amazon Connect API from downstream exhaustion.</li> <li>• Cap maximum concurrent contact center payloads to matches downstream threshold.</li> </ul> | <ul style="list-style-type: none"> <li>• AWS Lambda Concurrency Limits</li> <li>• CloudWatch Alarms (Throttling metrics)</li> </ul>                                 | AWS Lambda<br>Amazon Connect                            |

## Appendix B: Cost Optimization & Attribution Matrix

This appendix provides a structured evaluation framework for linking operational expenses to their true business drivers, ensuring financial accountability and resource efficiency

| Cost Domain                            | Identified Bleed Point                                                                                                                                                                | Architectural Fix / Optimization Strategy                                                                                                                                                                                                                           | Expected Financial Impact                                                                                                                                                |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compute Logic</b>                   |  CapEx<br> OpEx     | <ul style="list-style-type: none"><li>Implement AWS Compute Optimizer to automatically tune Lambda memory-to-runtime execution ratios.</li><li>Leverage Amazon AppFlow for native data syncing, minimizing custom polling code development.</li></ul>               | <ul style="list-style-type: none"><li>OpEx: Reductions of 15% to 30% in Lambda execution costs.</li><li>CapEx: Drops in integration development effort.</li></ul>        |
| <b>Data Storage &amp; State</b>        |  CapEx<br> OpEx | <ul style="list-style-type: none"><li>Enforce S3 Lifecycle rules to push 90-day logs to Glacier Deep Archive.</li><li>Convert DynamoDB tables to On-Demand (Pay-per-Request) scaling and apply Time-To-Live (TTL).</li></ul>                                        | <ul style="list-style-type: none"><li>OpEx: Savings of up to 70% on stale storage.</li><li>CapEx: Zero-cost idle environment footprints.</li></ul>                       |
| <b>Data Transfer &amp; Integration</b> |  CapEx<br> OpEx | <ul style="list-style-type: none"><li>Switch payload pipelines to Salesforce Change Data Capture (CDC) or Platform Events via AppFlow to stream batched deltas.</li><li>Replace third-party integration platforms with native Amazon EventBridge routing.</li></ul> | <ul style="list-style-type: none"><li>OpEx: Salesforce API call counts plunge by up to 60%.</li><li>CapEx: Reduced dependency on third-party licenses.</li></ul>         |
| <b>Observability &amp; Logging</b>     |  CapEx<br> OpEx | <ul style="list-style-type: none"><li>Set CloudWatch log retention periods to 14 days</li><li>Implement a dynamic Log-Level controller (via Lambda Environment Variables) to use debug mode only during active troubleshooting.</li></ul>                           | <ul style="list-style-type: none"><li>OpEx: CloudWatch log ingestion costs cut by 40% to 50%.</li><li>CapEx: Avoided enterprise platform contract commitments.</li></ul> |

|                                      |                                                                                                                                                                                       |                                                                                                                                                                                                                                                                          |                                                                                                                                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Management &amp; Gateways</b> |  CapEx<br> OpEx     | <ul style="list-style-type: none"> <li>Implement API Gateway Caching for static responses.</li> <li>Offload heavy payload validation directly to Amazon API Gateway Request Validators before hitting Lambdas.</li> </ul>                                                | <ul style="list-style-type: none"> <li>OpEx: Reduce downstream compute triggers by up to 35%.</li> <li>CapEx: Eliminates third-party API firewall subscription costs.</li> </ul>          |
| <b>Event Routing &amp; Messaging</b> |  CapEx<br> OpEx     | <ul style="list-style-type: none"> <li>Implement precise EventBridge Event Patterns to filter and drop irrelevant events at the bus edge.</li> <li>Use native EventBridge Dead Letter Queues (DLQs).</li> </ul>                                                          | <ul style="list-style-type: none"> <li>OpEx: Lowers EventBridge processing fees by 20% to 40%.</li> <li>CapEx: Cuts pipeline delivery development sprint cycles.</li> </ul>               |
| <b>Security &amp; Key Management</b> |  CapEx<br> OpEx    | <ul style="list-style-type: none"> <li>Enable AWS KMS Encryption Context and Data Key Caching within the integration SDK.</li> <li>Leverage AWS-managed keys where compliance dictates, and Customer Managed Keys only for sensitive data fields.</li> </ul>             | <ul style="list-style-type: none"> <li>OpEx: KMS API call expenses decline by up to 80%.</li> <li>CapEx: Zero upfront hardware security infrastructure investment.</li> </ul>             |
| <b>Testing &amp; Environments</b>    |  CapEx<br> OpEx | <ul style="list-style-type: none"> <li>Deploy testing environments dynamically via Infrastructure as Code (IaC) pipelines.</li> <li>Tear down sandboxes automatically on weekends using AWS CloudControl or Scheduled Lambda scripts.</li> </ul>                         | <ul style="list-style-type: none"> <li>OpEx: Non-production cloud operational spend drops by 30%.</li> <li>CapEx: Eradicates fixed-capacity infrastructure lock-in.</li> </ul>            |
| <b>Notification Services</b>         |  CapEx<br> OpEx | <ul style="list-style-type: none"> <li>Implement SNS Message Delivery Status logging to trace and eliminate failed endpoints.</li> <li>Add an upstream Lambda aggregator to batch alerts into single emails or Slack webhooks during high-frequency failures.</li> </ul> | <ul style="list-style-type: none"> <li>OpEx: Prevent compounding utility spend spikes by up to 50%.</li> <li>CapEx: Shifts development to low-cost cloud communication models.</li> </ul> |

|                                            |                                                                                                                                                                                                   |                                                                                                                                                                                                                                             |                                                                                                                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Secrets<br/>Credential<br/>Rotation</b> | <b>&amp;</b><br> CapEx<br> OpEx | <ul style="list-style-type: none"> <li>• Implement in-memory token caching within the global scope of AWS Lambda functions.</li> <li>• Re-fetch tokens from Secrets Manager only upon cache expiration (e.g., every 15 minutes).</li> </ul> | <ul style="list-style-type: none"> <li>• OpEx: Secrets Manager API requests plummet by 95% or more.</li> <li>• CapEx: Lowers security remediation and compliance overhead.</li> </ul> |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Appendix C: Enterprise Security Risk Register

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

| Security Vulnerability Description                                                                                                                                                                 | Likelihood (1-5) | Impact (1-5) | Risk Rating (1-25) | Core Technical Mitigation Strategy                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|--------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Salesforce Session Token Exhaustion / Exfiltration</b><br>Lambda code fetches Salesforce OAuth tokens on every execution, causing token leakage in logs or API rate-limiting denial of service. | 3                | 4            | 12 (Medium)        | Implement AWS Secrets Manager with forced in-memory caching in the Lambda global scope. Configure automated rotation using AWS Lambda secret rotation hooks. |
| <b>Data in Transit Interception via Legacy Protocols</b><br>Man-in-the-middle (MITM) attacks intercepting raw customer payloads passing between Salesforce Webhooks/AppFlow and AWS.               | 2                | 5            | 10 (Medium)        | Enforce TLS 1.3 across all API Gateway endpoints and AppFlow connections. Deploy AWS WAF (Web Application Firewall) to block non-compliant traffic.          |
| <b>Lambda Resource Over-Privilege (IAM Blast Radius)</b><br>Lambda functions sharing a single broad IAM role, allowing an exploited function to modify unrelated DynamoDB tables or S3 buckets.    | 4                | 4            | 16 (High)          | Enforce strict Least-Privilege IAM Roles per Lambda function. Use AWS IAM Access Analyzer to audit and eliminate wildcard (*) resource permissions.          |
| <b>DynamoDB Injection / Data Extrapolation</b><br>Unsanitized input from Salesforce fields manipulating AWS Lambda downstream queries, leading to unauthorized data exposure.                      | 3                | 4            | 12 (Low)           | Use DynamoDB Expression Attribute Values and placeholders for all queries. Never construct raw execution strings using direct string concatenation.          |
| <b>Unprotected Endpoints / Webhook Spoofing</b>                                                                                                                                                    | 4                | 4            | 16 (High)          | Implement Salesforce Mutual Authentication                                                                                                                   |



|                                                                                                                                                                                                                                                               |   |   |                |                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attackers bypassing Salesforce to send forged HTTP POST requests directly to the AWS API Gateway endpoint.                                                                                                                                                    |   |   |                | (mTLS) or require digital signatures validated against a public key stored securely in AWS Systems Manager.                                                                              |
| <b>Credential Hardcoding in Source Control</b><br>Developers embedding Salesforce Client Secrets or AWS Access Keys directly inside Lambda codebase repositories.                                                                                             | 3 | 5 | 15<br>(Medium) | Integrate git-secrets or AWS CodeGuru Security scanners into the CI/CD pipeline. Block any commit containing patterns resembling private keys.                                           |
| <b>Log Clearing / Traceability Deletion</b><br>A breach of the cloud-tier master encryption keys exposes localized application states, data stores, and historical logging files.                                                                             | 2 | 4 | 8<br>(Low)     | Stream CloudWatch logs to a central security account S3 bucket secured with S3 Object Lock in Compliance Mode to guarantee immutability.                                                 |
| <b>Event Spoofing via EventBridge Target Pollution</b><br>An attacker manipulating internal system components to publish malicious events directly onto the custom EventBridge bus, bypassing Salesforce validation logic.                                    | 2 | 4 | 8<br>(Low)     | Enforce strict EventBridge Bus Resource Policies that explicitly restrict PutEvents actions to verified IAM roles. Implement schema validation using Amazon EventBridge Schema Registry. |
| <b>Salesforce Restrictive IP Bypass via Lambda Ephemeral Ips</b><br>Standard AWS Lambdas utilizing dynamic public IPs to write back to Salesforce, exposing the Salesforce API perimeter to traffic coming from outside trusted corporate corporate networks. | 3 | 4 | 12<br>(Medium) | Route all outbound AWS Lambda traffic through an AWS NAT Gateway with allocated Elastic IPs. Register these static IP addresses within the Salesforce Network Access Allowed IP Ranges   |
| <b>Downstream Denial of Wallet via Event Loop Poisoning</b><br>Maliciously structured or infinitely looping payloads from                                                                                                                                     | 3 | 3 | 9<br>(Medium)  | Configure Maximum Age of Record and Maximum Retry Attempts (set to 2) on the Lambda Event Source Mapping. Immediately                                                                    |

|                                                                                                                     |  |  |  |                                                            |
|---------------------------------------------------------------------------------------------------------------------|--|--|--|------------------------------------------------------------|
| Salesforce causing AWS Lambdas to retry infinitely, exhausting budget caps and locking system compute availability. |  |  |  | offload poisoned events to an SQS Dead Letter Queue (DLQ). |
|---------------------------------------------------------------------------------------------------------------------|--|--|--|------------------------------------------------------------|

## Appendix D: Enterprise RACI Matrix

This matrix defines the clear operational boundaries, decision rights, and delivery expectations across cross-functional teams to eliminate role confusion and accelerate project execution.

| <b>Key Architectural Deliverable</b>                    | Salesforce Admin / Dev Team | Cloud Platform Eng. (AWS) | Cyber Security & GRC | DevSecOps / CI/CD Team | Enterprise Architecture |
|---------------------------------------------------------|-----------------------------|---------------------------|----------------------|------------------------|-------------------------|
| Defining API Integrations & Event Schemas               | A                           | R                         | C                    | I                      | C                       |
| Provisioning AWS Serverless Stack (Lambda, EventBridge) | I                           | A                         | C                    | R                      | I                       |
| Configuring AppFlow Connections & Salesforce CDC        | R                           | A                         | C                    | I                      | I                       |
| IAM Role & Least-Privilege Policy Definition            | I                           | R                         | A                    | R                      | C                       |
| KMS Key Management & Data Encryption Policies           | I                           | R                         | A                    | I                      | C                       |
| Idempotency & Payload Validation Code Logic             | C                           | A                         | C                    | I                      | C                       |
| CI/CD Pipeline Construction & IaC Code Scanning         | I                           | C                         | C                    | A                      | I                       |
| CloudWatch Alarm Thresholds & SNS Alerting Setup        | I                           | A                         | I                    | R                      | I                       |
| Disaster Recovery (DR) & Failover Execution             | C                           | A                         | C                    | R                      | C                       |
| Salesforce API Limit Monitoring & Governance            | A                           | C                         | I                    | I                      | I                       |
| Cost Allocation, Budgeting, & Tagging Enforcement       | I                           | R                         | I                    | R                      | A                       |

|                                                            |          |          |          |          |          |
|------------------------------------------------------------|----------|----------|----------|----------|----------|
| Incident Response<br>for Poison Payloads<br>/ DLQ Failures | <i>R</i> | <i>R</i> | <i>C</i> | <i>I</i> | <i>I</i> |
|------------------------------------------------------------|----------|----------|----------|----------|----------|

# About the Author

## Amit Kulkarni

*Senior Solutions Architect*

📍 Pune, India | ✉ [amitkwork2920@gmail.com](mailto:amitkwork2920@gmail.com) | 🔗 [LinkedIn](#) | 💻 [Github](#)

I'm a Technology-agnostic architecture leader having 18+ years of partnership with 15+ clients across BFSI, Retail, Insurance, Healthcare, Travel, Media, and the Public Sector. Proven track record in cloud transformation (AWS, Azure), legacy COTS and mainframe modernization, security posture assessments, microservices modernization, event-driven architectures, and working knowledge on AI initiatives.

### Architectural Specialization (Recent Focus)

During a recent deliberate transition period away from formal corporate employment, I dedicated my time to advanced deep-dives into enterprise cloud topologies, edge cases, and failure domains. This case study on *Legacy Salesforce Hybrid Modernization on AWS* is part of a broader body of architectural research aimed at critiquing standard vendor blueprints, identifying hidden system vulnerabilities, and building production-ready engineering remediations for high-throughput transactional environments.

I am currently seeking my next high-impact engineering role where I can solve complex distributed systems challenges and help scale robust, mission-critical infrastructure teams.